

Numerical Questions:

1. Memory Calculation for a Struct with Alignment

Struct Definition:

C

Copy code

```
struct SensorData {
    char id;
    int value;
    short status;
};
```

Assumptions:

- `char` is 1 byte.
- `int` is 4 bytes (requires 4-byte alignment).
- `short` is 2 bytes.

Memory Layout (4-byte alignment):

1. `id` (1 byte) → Requires 3 bytes of padding to align `value` (next member) to a 4-byte boundary.
2. `value` (4 bytes) → Already aligned.
3. `status` (2 bytes) → Requires 2 bytes of padding at the end to align the total struct size to a multiple of 4 bytes.

Total Memory Size:

- $1 \text{ (id)} + 3 \text{ (padding)} + 4 \text{ (value)} + 2 \text{ (status)} + 2 \text{ (padding)} = 12 \text{ bytes.}$

Explanation: Padding ensures proper alignment, which is critical for ARM Cortex-M processors to avoid unaligned memory access and maintain performance.

2. Assembly Loop Calculation

Task: Sum integers from 1 to `n` using ARM assembly, where `n = 5`.

Assembly Code:

assembly

Copy code

```
MOV r0, #0      ; sum = 0
MOV r1, #1      ; i = 1
MOV r2, #5      ; n = 5
```

loop:

```
ADD r0, r0, r1  ; sum += i
ADD r1, r1, #1  ; i++
```

```
CMP r1, r2      ; compare i with n + 1
BNE loop        ; if i <= n, branch to loop
```

; r0 now contains the sum

Step-by-Step Execution:

- Iteration 1: $r0 = 0 + 1 = 1$, $r1 = 2$.
- Iteration 2: $r0 = 1 + 2 = 3$, $r1 = 3$.
- Iteration 3: $r0 = 3 + 3 = 6$, $r1 = 4$.
- Iteration 4: $r0 = 6 + 4 = 10$, $r1 = 5$.
- Iteration 5: $r0 = 10 + 5 = 15$, $r1 = 6$.

Result: The register `r0` contains **15** after the loop.

3. Static Variable Increment

C Code:

```
c
Copy code
int count_calls() {
    static int count = 0;
    count += 1;
    return count;
}
```

Return Values:

1. First call: `count = 1`.
2. Second call: `count = 2`.
3. Third call: `count = 3`.

Explanation: The static variable `count` is stored in global memory, retaining its value between function calls.

ARM Assembly Equivalent:

```
assembly
Copy code
LDR r0, =count      ; Load the address of the static variable
LDR r1, [r0]         ; Load current value of count
ADD r1, r1, #1       ; Increment count
STR r1, [r0]         ; Store updated value back to memory
MOV r0, r1           ; Return the updated count
BX lr                ; Return from function
```

4. Unaligned Access Cost Calculation

Unaligned Address: `0x20000001`.

Solution:

1. Read each byte individually:
 - Byte 1: `LDRB r0, [r1]` (Address: `0x20000001`).
 - Byte 2: `LDRB r1, [r1, #1]` (Address: `0x20000002`).
 - Byte 3: `LDRB r2, [r1, #2]` (Address: `0x20000003`).
 - Byte 4: `LDRB r3, [r1, #3]` (Address: `0x20000004`).

Reassemble bytes into a 32-bit value:

assembly

Copy code

```
ORR r0, r0, LSL #24
ORR r0, r1, LSL #16
ORR r0, r2, LSL #8
ORR r0, r3
```

2. Access Time:

- 4 memory accesses $\times$ 1 clock cycle per access = **4 clock cycles**.
-

5. Optimize a Mixed C-Assembly Program for Summing an Array

C Code:

c

Copy code

```
int sum_array(int *arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    return sum;
}
```

ARM Assembly Code:

assembly

Copy code

```
MOV r0, #0      ; sum = 0
MOV r1, #0      ; i = 0
```

loop:

```
LDR r2, [r3, r1] ; Load arr[i]
ADD r0, r0, r2   ; sum += arr[i]
ADD r1, r1, #4   ; i++
CMP r1, r4       ; Compare i with n
BNE loop         ; If i < n, continue loop
```

```
; r0 now contains the sum
```

Cycle Count:

- **C Code:** 10 iterations × 4 cycles = **40 cycles**.
- **Assembly Code:** 10 iterations × 2 cycles = **20 cycles**.

Saved Cycles: 20 cycles.

Practical Application Questions:

1. Accessing Static Data from Assembly

C Code:

```
c
Copy code
int counter = 0;
void increment_counter() {
    counter++;
}
```

ARM Assembly Code:

```
assembly
Copy code
.extern counter
LDR r0, =counter
LDR r1, [r0]
ADD r1, r1, #1
STR r1, [r0]
BX lr
```

Explanation: The `.extern` directive allows the assembly code to reference the `counter` variable defined in C. After three calls, `counter = 3`.

2. Mixing C and Assembly in Data Processing

Task: Find the maximum value in an array.

ARM Assembly Code:

```
assembly
Copy code
MOV r0, #0          ; max = 0
MOV r1, #5          ; n = 5
LDR r2, =readings   ; Load address of array

loop:
LDR r3, [r2], #4     ; Load next element
CMP r3, r0           ; Compare with max
MOVGT r0, r3         ; Update max if r3 > r0
SUBS r1, r1, #1      ; Decrement n
BNE loop            ; Continue loop if n > 0

; r0 contains the max value
```

3. Unaligned Data Access Issue

Solution: Read data byte-by-byte and reassemble:

assembly

Copy code

```
LDRB r0, [r1]      ; Read byte 1
LDRB r2, [r1, #1]   ; Read byte 2
LDRB r3, [r1, #2]   ; Read byte 3
LDRB r4, [r1, #3]   ; Read byte 4

ORR r0, r0, LSL #24
ORR r0, r2, LSL #16
ORR r0, r3, LSL #8
ORR r0, r4
```

4. Memory Alignment for Performance

Struct Definition:

c

Copy code

```
struct ControlData {
    int command;
    short status;
    char flag;
};
```

Size (Aligned): 12 bytes (2 bytes padding after `status` + 1 byte padding after `flag`).

Size (Packed): 7 bytes (no padding).

Trade-off: Using `__packed` reduces memory usage but increases access time due to unaligned accesses.

5. Calling Assembly from C for Device Control

C Code:

c

Copy code

```
extern void toggle_led();
toggle_led();
```

Assembly Code:

assembly

Copy code

```
toggle_led:
    LDR r0, =GPIO_BASE
    LDR r1, [r0, #LED_PIN]
    EOR r1, r1, #1
    STR r1, [r0, #LED_PIN]
```

`BX lr`

Timing Improvement: Assembly directly manipulates GPIO registers with precise control, which is faster than equivalent C code due to reduced overhead.

Numerical Questions

1. Stack Pointer Adjustment for Function Calls

When a function in ARM Cortex-M is called, the stack pointer (SP) adjusts to store the return address, local variables, and parameters.

Question:

Consider the following ARM function:

assembly

Copy code

foo:

```
SUB sp, sp, #12      ; Allocate 12 bytes for local variables
ADD r0, r1, r2       ; Perform some operation
ADD sp, sp, #12      ; Restore the stack pointer
BX lr               ; Return from function
```

If the stack pointer starts at `0x20001000`, calculate its value:

1. After the function prologue (first `SUB` instruction).
2. After the function epilogue (after the `ADD` instruction).

1. Stack Pointer Adjustment for Function Calls

Question Recap: If the stack pointer (SP) starts at `0x20001000`, calculate its value:

1. After the function prologue (`SUB sp, sp, #12`).
2. After the function epilogue (`ADD sp, sp, #12`).

Solution:

1. **Prologue (`SUB sp, sp, #12`):**
 - The stack pointer decreases to allocate 12 bytes for local variables.
 - New SP = Old SP - 12 = `0x20001000` - `0xC` = `0x20000FF4`.
2. **Epilogue (`ADD sp, sp, #12`):**
 - The stack pointer is restored to its original value by adding back 12 bytes.
 - Restored SP = `0x20000FF4` + `0xC` = `0x20001000`.

Explanation: The stack pointer must be adjusted to allocate space for local variables during the function execution. When the function returns, SP is restored to its original value to maintain stack integrity.

2. Function Call Overhead in C and Assembly

When calling a function in C, the ARM Cortex-M pushes registers to the stack.

Question:
Suppose a function takes three arguments and is called in C as follows:

```
c
Copy code
int result = my_function(a, b, c);
```

If ARM Cortex-M uses registers `r0`, `r1`, and `r2` for the arguments, calculate:

1. How much memory is used on the stack for storing registers during the function call?
2. How many cycles are saved by writing the function in assembly if you avoid unnecessary register pushes/pops?

2. Function Call Overhead in C and Assembly

Question Recap: If a function takes three arguments (stored in `r0`, `r1`, `r2`), calculate:

1. Memory used on the stack for register storage.
2. Cycles saved by avoiding unnecessary pushes/pops in assembly.

Solution:

1. **Memory Usage:**
 - For function calls, ARM Cortex-M saves the return address (`lr`) and additional registers (`r4-r11`) if needed.
 - Assuming only `lr` is stored, memory used = 4 bytes (return address).
2. **Cycle Savings:**
 - In C, additional cycles are used for saving/restoring registers.
 - Avoiding these saves:
 - Push cycles: 2 per register
 - Pop cycles: 2 per register Total = 4 cycles × (1 saved register) = 4 cycles saved.

3. Memory Layout and Data Alignment

Consider this struct:

```
c
Copy code
struct MotorControl {
    char mode;
    short speed;
    int position;
};
```

Question:

1. Calculate the size of the struct with and without alignment on ARM Cortex-M (4-byte alignment for `int`).
2. Explain how padding is added and its impact on performance.

3. Memory Layout and Data Alignment

Question Recap: Calculate the size of the struct:

```
c
Copy code
```

```
struct MotorControl {
    char mode;
    short speed;
    int position;
};
```

Solution:

1. Field Sizes and Alignment:

- `char mode`: 1 byte, aligned to 1 byte.
- `short speed`: 2 bytes, aligned to 2 bytes.
- `int position`: 4 bytes, aligned to 4 bytes.

2. Padding Calculation:

- `mode` (1 byte) → Add 1 byte padding to align `speed`.
- `speed` (2 bytes) → No padding needed.
- `position` (4 bytes) → Add 2 bytes padding after `speed` for alignment.

3. Total Size = 1 (mode) + 1 (padding) + 2 (speed) + 2 (padding) + 4 (position) = **12 bytes**.

Explanation: Padding ensures data fields are aligned for faster access. Misaligned fields require multiple memory accesses, impacting performance.

Great! Let's start with detailed solutions for the **numerical and practical application questions** related to **mixing C and Assembly on ARM Cortex-M microcontrollers**. I'll provide step-by-step explanations for each.

3. Memory Layout and Data Alignment

Question Recap: Calculate the size of the struct:

```
c
Copy code
struct MotorControl {
    char mode;
    short speed;
    int position;
};
```

Solution:

1. Field Sizes and Alignment:

- `char mode`: 1 byte, aligned to 1 byte.
- `short speed`: 2 bytes, aligned to 2 bytes.
- `int position`: 4 bytes, aligned to 4 bytes.

2. Padding Calculation:

- `mode` (1 byte) → Add 1 byte padding to align `speed`.
- `speed` (2 bytes) → No padding needed.
- `position` (4 bytes) → Add 2 bytes padding after `speed` for alignment.

3. Total Size = 1 (mode) + 1 (padding) + 2 (speed) + 2 (padding) + 4 (position) = **12 bytes**.

Explanation: Padding ensures data fields are aligned for faster access. Misaligned fields require multiple memory accesses, impacting performance.

4. Loop Optimization in Assembly

Question Recap: Optimize the C factorial function into ARM assembly and calculate cycle savings for $n = 5$.

Solution:

C Code:

C

Copy code

```
int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Equivalent ARM Assembly:

assembly

Copy code

```
factorial:
    MOV r1, #1          ; result = 1
    MOV r2, #1          ; i = 1
loop:
    MUL r1, r1, r2      ; result *= i
    ADD r2, r2, #1      ; i++
    CMP r2, r0          ; Compare i with n
    BGT end_loop       ; Exit loop if i > n
    B loop              ; Repeat
end_loop:
    MOV r0, r1          ; Return result
    BX lr               ; Return to caller
```

Cycle Calculation for $n = 5$:

- **C Implementation:** 4 cycles per iteration $\times$ 5 iterations = **20 cycles**.
- **Assembly Implementation:** 2 cycles per iteration $\times$ 5 iterations = **10 cycles**.
- **Savings:** $20 - 10 = 10$ cycles saved.

5. Static Array Access and Cache Behavior

Question Recap: Write assembly code to sum an array, and address misalignment issues.

Solution:

ARM Assembly Code for Summing Array:

assembly

Copy code

```
sum_array:
    MOV r2, #0           ; sum = 0
    MOV r3, #0           ; i = 0
loop:
    LDR r1, [r0, r3]      ; Load arr[i]
    ADD r2, r2, r1        ; sum += arr[i]
    ADD r3, r3, #4        ; Move to next element (4 bytes)
    CMP r3, r1            ; Compare i with n
    BNE loop              ; Continue loop if i < n
end_loop:
    MOV r0, r2            ; Return sum
    BX lr
```

Addressing Misalignment:

If the array starts at 0x20000003, unaligned access will require reading each byte and reassembling:

assembly

Copy code

```
LDRB r1, [r0]           ; Read byte 0
LDRB r2, [r0, #1]       ; Read byte 1
LDRB r3, [r0, #2]       ; Read byte 2
LDRB r4, [r0, #3]       ; Read byte 3
ORR r1, r1, LSL #8      ; Combine bytes
ORR r1, r1, r3, LSL #16
ORR r1, r1, r4, LSL #24
```
